

M04 - Python Foundations

16-lesson beginner-sufficient learner book - RC1

How to use each lesson

Read the explanation, reproduce the small example, complete the matching guided lab, then close the book for the independent task. A lesson is not mastered merely because the example works.

DE04-01 - Python setup and first program

Why this matters	You need a reproducible way to run small scripts and inspect failures.
------------------	--

Core explanation

- Run Python from a known environment.
- A script is plain text executed by the interpreter.
- Syntax errors stop parsing before runtime.
- Save the exact error before fixing it.

Concrete example

```
print('hello data engineering')
```

Common failure	Do not copy a command you cannot explain which interpreter runs.
----------------	--

Proof before move-on	Run a script, create one syntax error intentionally, then fix it.
----------------------	---

Explain aloud

In 60-90 seconds explain python setup and first program, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-02 - Variables, data types and operators

Why this matters	Programs store named values and combine them with explicit operations.
------------------	--

Core explanation

- Use descriptive variable names.
- int, float, str and bool have different behavior.
- Operators can calculate and compare.
- Check types when debugging surprising results.

Concrete example

```
sales = qty * unit_price; is_large = sales >= 3000
```

Common failure	Do not concatenate numeric strings when you intended addition.
Proof before move-on	Calculate order sales and print value plus type.

Explain aloud

In 60-90 seconds explain variables, data types and operators, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-03 - Strings and formatting

Why this matters	Text cleaning and readable output are common data tasks.
------------------	--

Core explanation

- strip removes surrounding whitespace.
- lower/title can standardize display cautiously.
- f-strings build readable messages.
- Strings are immutable values.

Concrete example

```
region = '  NORTH zone '.strip().title()
```

Common failure	Do not normalize identifiers in a way that changes their meaning.
Proof before move-on	Clean a label and build a readable output string.

Explain aloud

In 60-90 seconds explain strings and formatting, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-04 - Conditions and boolean logic

Why this matters	Branching turns business rules into explicit code paths.
------------------	--

Core explanation

- if/elif/else evaluate conditions in order.
- and/or/not combine booleans.
- Keep complex rules readable.
- Test boundary values.

Concrete example

high_value = status == 'Completed' and sales >= 2500

Common failure	Do not forget inclusive/exclusive threshold boundaries.
Proof before move-on	Classify three orders including an exact-threshold case.

Explain aloud

In 60-90 seconds explain conditions and boolean logic, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-05 - Lists, tuples, sets and dictionaries

Why this matters	Choose collections based on ordering, uniqueness and lookup needs.
------------------	--

Core explanation

- Lists are ordered and mutable.
- Tuples are ordered and immutable by convention.
- Sets hold unique hashable values.
- Dictionaries map keys to values.

Concrete example

```
manager_by_region = {'East': 'Asha', 'West': 'Ravi'}
```

Common failure	Do not use a list scan repeatedly when a dictionary/set expresses the access pattern better.
Proof before move-on	Deduplicate IDs and retrieve one manager by region.

Explain aloud

In 60-90 seconds explain lists, tuples, sets and dictionaries, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-06 - Loops and iteration

Why this matters

Loops process repeated items while keeping state explicit.

Core explanation

- for iterates over a collection.
- while repeats while a condition remains true.
- Accumulators build totals/counts.
- Avoid infinite loops by changing the loop condition.

Concrete example

```
total = 0
for value in sales_values:
    total += value
```

Common failure

Do not modify a list unpredictably while iterating over it.

Proof before move-on

Sum values without sum() and count values above a threshold.

Explain aloud

In 60-90 seconds explain loops and iteration, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-07 - Functions and scope

Why this matters	Functions create reusable, testable units with explicit inputs and outputs.
------------------	---

Core explanation

- Parameters are inputs.
- `return` produces an output value.
- Local variables belong to the function call.
- Small pure functions are easier to test.

Concrete example

```
def order_label(order_id, region):  
    return f'{order_id} | {region}'
```

Common failure	Do not hide required inputs in mutable globals.
----------------	---

Proof before move-on	Write and test a function with two different inputs.
----------------------	--

Explain aloud

In 60-90 seconds explain functions and scope, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-08 - Modules, imports and project structure

Why this matters	Splitting code into files makes responsibilities clearer.
------------------	---

Core explanation

- A module is a Python file importable by name.
- Imports should not trigger destructive work.
- Use an entry-point guard for executable scripts.
- Keep helpers separate from orchestration.

Concrete example

```
if __name__ == '__main__':  
    main()
```

Common failure	Do not put the whole pipeline at module import time.
----------------	--

Proof before move-on	Complete a helper function and call it from another module.
----------------------	---

Explain aloud

In 60-90 seconds explain modules, imports and project structure, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-09 - Exceptions and debugging basics

Why this matters	Errors are evidence. Reading tracebacks is a core engineering skill.
------------------	--

Core explanation

- Tracebacks show call stack and failing line.
- TypeError, ValueError, KeyError and FileNotFoundError mean different things.
- Catch only errors you can handle meaningfully.
- Log context without secrets.

Concrete example

```
try:
    amount = float(raw_amount)
except ValueError:
    ...
```

Common failure	Do not wrap everything in bare except: and silently continue.
Proof before move-on	Run the debug file, record each failure, then repair it.

Explain aloud

In 60-90 seconds explain exceptions and debugging basics, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-10 - Files, paths, CSV and JSON I/O

Why this matters

Data engineering code must read and write files predictably.

Core explanation

- Use pathlib for clear paths.
- Specify text encoding where relevant.
- CSV is tabular text; JSON preserves nested structure.
- Separate input/output directories and avoid overwriting raw data.

Concrete example

```
from pathlib import Path
input_path = Path('data') / 'orders.csv'
```

Common failure

Do not assume the current working directory is always the script directory.

Proof before move-on

Read a CSV/JSON sample and write an output to a separate path.

Explain aloud

In 60-90 seconds explain files, paths, csv and json i/o, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-11 - pandas foundations: load, inspect, filter and aggregate

Why this matters	pandas is useful for small/medium local data and validation tasks.
------------------	--

Core explanation

- read_csv loads tabular data.
- head/info/dtypes expose structure.
- Boolean masks filter rows.
- groupby + agg summarizes by keys.

Concrete example

```
df[df['status']=='Completed'].groupby('region')['sales'].sum()
```

Common failure	Do not trust inferred dtypes without inspection.
Proof before move-on	Calculate completed sales by region and reconcile to a direct sum.

Explain aloud

In 60-90 seconds explain pandas foundations: load, inspect, filter and aggregate, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-12 - Practical Big-O intuition

Why this matters

You need enough complexity intuition to avoid obviously poor repeated work.

Core explanation

- $O(1)$ describes constant-time-style access patterns.
- $O(n)$ scans grow with input size.
- Nested scans can approach $O(n^2)$.
- Data-engineering performance is also dominated by I/O and distributed shuffles.

Concrete example

Checking membership in a set is usually a better repeated-lookup pattern than rescanning a list.

Common failure

Do not turn Big-O into fake precision about real runtime.

Proof before move-on

Explain one list-vs-set choice in plain English.

Explain aloud

In 60-90 seconds explain practical big-o intuition, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-13 - Coding drills: lists and strings

Why this matters

Short drills build interview fluency without replacing project work.

Core explanation

- Index/slice carefully.
- Prefer readable built-ins when allowed.
- Handle empty inputs.
- Write a small test for each function.

Concrete example

Normalize names, find the longest string, and reverse a list without corrupting the original.

Common failure

Do not optimize a 5-line drill into unreadable code.

Proof before move-on

Complete three fresh list/string functions without notes.

Explain aloud

In 60-90 seconds explain coding drills: lists and strings, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-14 - Coding drills: dictionaries and sets

Why this matters	Hash-based collections appear in dedupe, counts and lookups.
------------------	--

Core explanation

- Use dict.get for counting patterns.
- Use sets for seen/duplicate detection.
- Choose stable output order when tests require it.
- Document key assumptions.

Concrete example

Return duplicate order IDs only once while preserving first duplicate discovery order.

Common failure	Do not rely on unordered presentation when order is part of the contract.
----------------	---

Proof before move-on	Build region counts and a duplicate-ID finder.
----------------------	--

Explain aloud

In 60-90 seconds explain coding drills: dictionaries and sets, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-15 - Stack, queue, search and sort basics

Why this matters

Lightweight data structures protect interview fluency.

Core explanation

- A stack is LIFO.
- A queue is FIFO.
- Linear search is simple and $O(n)$.
- `sorted` returns a new list; `list.sort` mutates in place.

Concrete example

Use a list as a simple stack for [extract, transform, load], then pop once.

Common failure

Do not over-focus on DSA at the expense of SQL/pipelines.

Proof before move-on

Implement `linear_search` and explain one FIFO data-system analogy.

Explain aloud

In 60-90 seconds explain stack, queue, search and sort basics, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE04-16 - Integrated Python checkpoint and reproducible script habits

Why this matters	This lesson joins syntax, files, pandas, functions and debugging into one small utility.
------------------	--

Core explanation

- Separate read, validate, transform and report functions.
- Use explicit input/output paths.
- Return nonzero exit behavior for unrecoverable failures later in M05.
- Save environment/dependency notes.

Concrete example

Build a script that reads orders CSV, validates required columns, computes sales and writes region summary CSV.

Common failure	Do not solve everything in one top-level block.
Proof before move-on	Complete the module practical, then recreate the core script from a clean folder.

Explain aloud

In 60-90 seconds explain integrated python checkpoint and reproducible script habits, the input/output grain or program state, and one way a plausible-looking result could still be wrong.